MixBytes()

[SECURITY AUDIT REPORT]

Shift DeFi Aave V3 Strategies and CoW Protocol Integration

PREPARED FOR SHIFT DEFI

REPORT DATE

October 15, 2026

[CONTENTS]

Table of Contents

1. Introduction 3
   - 1.1 Disclaimer 3
   - 1.2 Executive Summary 3
   - 1.3 Project Overview 6
   - 1.4 Security Audit Methodology 9
   - 1.5 Risk Classification 11
   - 1.6 Summary of Findings 12
2. Findings Report 12
   - 2.1 Critical 13
   - 2.2 High 13
   - 2.3 Medium 14
     - M-1 MorphoVault reinvestment stores a pre-fee baseline, so every reinvesting harvest undercharges the fee 14
   - 2.4 Low 15
     - L-1 A 1-wei USDC donation makes harvest's Aave supply revert, stalling Vault batches 15
     - L-2 aToken fee transfer reverts during an Aave pause, blocking exits from the token state 16
     - L-3 At a 100% fee, harvest keeps a pre-fee checkpoint, exempting later income 17
     - L-4 Partial fill leaves input and router allowance on the adapter for anyone to take 18
     - L-5 Order retirement needs sell-token calls, so an issuer freeze locks batches until an upgrade 19
     - L-6 ReshufflingGateway.blacklistToken requires a zero balance, so any 1-wei transfer blocks delisting 20
     - L-7 Native MON on SwapRouter02 makes the Uniswap adapter revert, stalling harvest and Vault batches 21
3. About MixBytes 22

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Shift DeFi is a batch-based yield aggregator. Users deposit a notion token into a share-based `Vault`, and containers allocate the funds to strategies built on the shared `StrategyTemplate`, locally or on another chain. Every deposit and withdrawal batch is priced on the net asset value (NAV) that the strategies report, so NAV accuracy decides how value is split between depositing, withdrawing and remaining users.

The audit covered four repositories. In `shift-defi-strategies`, `AaveV3SupplyBase`, `AaveV3Supply` and `AaveV3SupplyWithMerkle` supply USDC to Aave V3 and charge the performance fee on aToken growth, and `AaveV3SupplyWithMerkle` also claims Merkl rewards, swaps them for USDC and reinvests them. In `shift-defi-swap-adapters`, `UniswapV3SwapRouter02Adapter` executes whitelisted Uniswap V3 paths through `SwapRouter02` for those swaps. In `shift-defi-cow-protocol-adapter`, `CowProtocolAdapter` places asynchronous CoW Protocol sell orders: it holds each pending order's sell tokens in its own `CowOrderLane` clone, which endorses the order through EIP-1271, and returns unspent tokens when the order is resolved or cancelled. In `shift-defi-platform`, the changes add these orders to the local and principal containers and the reshuffling gateway through `CowProtocolModule`, allow batch reports, strategy entries, reshuffling toggles and token delisting only when no order is pending, and let `StrategyTemplate` strategies enter a token state directly.

Two auditors reviewed the code over 4 working days, from 22 to 25 September 2026. The review combined manual line-by-line analysis, AI-assisted candidate generation with manual triage, fork tests against the live Aave V3, Uniswap V3 and CoW Protocol contracts and the live platform proxies, and stateful fuzzing of a container holding both Aave strategies.

Scope Limitations

The audit was limited to the files listed in the scope, and the platform and swap-adapter files were reviewed for their changes since the previously audited commits rather than audited in full. Tests, deployment scripts and configuration, external protocols and the other platform contracts were not audited; they were reviewed only to establish whether the in-scope code works correctly in its interactions with them.

Review Coverage

In addition to the audit team's general smart-contract security checklist, the review covered the following integration-specific questions:

- Is the performance fee charged only on accrued aToken growth, including across emergency exit, NAV resolution and donations?
- Does the strategy NAV include all value in every state, including after direct entry into a token state?
- Can Aave reserve limits or a failing reward route block enter and exit for longer than the event itself?
- Can the reward path or the adapter's permissionless `swap` leak value or reach principal, and do the limited roles stay within their mandate?
- Can lane tokens leave only to their own order's settlement or back to the owner, and is every fill classified correctly?
- Do lane occupancy, the pending set and order records stay consistent across placement, resolution, cancellation and sweeps?

- Are NAV reports, strategy entries and mode switches blocked while value is in flight, and can third parties gain leverage through order parameters or the off-chain orderbook?
- Do administrative paths remain executable under the new pending-order gates, and does upgrading the live proxies preserve their storage?

Deployment Requirements

- The live `ContainerLocal` and `ContainerPrincipal` proxies were deployed without the `uint256[50] private __gap` that `Container` now declares, so the reviewed build would shift every later variable by 50 slots and halt all batches. Their upgraded implementations must keep the deployed storage layout, with new state in namespaced storage or after the last deployed slot, and no variable may occupy `ContainerPrincipal` slots 13–14, which keep stale non-zero values.
- The containers and the reshuffling gateway of the upgraded `Vault` must run implementations with `pendingCowOrderCount()`, because the `Vault` calls it on them without a fallback.
- Each container with a CoW adapter set must have a `COW_SWAP_MANAGER_ROLE` holder.
- Every token traded through CoW must have a non-zero dust threshold, since a donation to a pending order's lane reaches the container inside a report.
- Only tokens that cannot rebase downwards may be sold through CoW: a lane whose balance falls below the signed sell amount cannot settle its order, which then blocks the pending-order gates until it is cancelled.
- Every Aave strategy must list `reserveAsset` among its output tokens.
- Each `SwapRouter` route through the Uniswap adapter must use a path that starts with the route's input token and ends with its output token, because the adapter rejects any other path and a reverting route reverts `harvest()`, and with it every enter and exit, even with `mustSucceed = false`.

Additional Attack Hypotheses

In addition to the issues described in this report, the audit examined the following protocol-specific attack hypotheses.

- Whether listing the notion token or the aToken as a reward token could make harvest sell principal. `_setRewardTokens` rejects both.
- Whether a batch could be priced while value is in flight through a flow without the pending-order gate. No such flow reads balances into a NAV figure or reaches lane-held tokens, and every path that reports a figure first resolves pending orders or reverts.
- `freeFilledAmountStorage` clears its settlement record. The verdict then relies on the lane's relayer allowance, which no path lowers without a pull for any whitelisted token.

Additional Notes

- Reward swaps run with a zero output floor. On Monad the block leaders see pending transactions, so private submission does not protect these swaps: on the whitelisted WMON/USDC pool a one-block sandwich takes most of a reward lump above about $3.5k, and Merkl rewards should be claimed often enough to keep each lump below that.
- The CoW placing role alone sets each order's limit price, `validTo` and `appData`, with no on-chain bounds, and the token manager can install any adapter that names the consumer as its owner, so both roles are trusted with the value of every order.

Overall, the new code is compact and follows the platform's existing structure: the Aave strategies build on `StrategyTemplate`, and the CoW integration keeps each order's tokens in its own lane until the order is resolved. The reported issues should be addressed and the Deployment Requirements above met before the Aave strategies are deployed and the CoW integration is activated.

After the interim report, the audit team re-audited the remediation commits to the strategies, swap-adapter and platform repositories against the reported findings. The Medium finding and five of the six Low findings are fixed. The CoW Protocol

adapter was not changed. The re-audit found one new Low issue in the updated swap adapter, which is fixed in a follow-up commit.

1.3 Project Overview

Summary

CLIENT	Shift
CATEGORY	Yield Aggregator
PROJECT	Aave V3 Strategies and CoW Protocol Integration
TYPE	Solidity
PLATFORM	EVM
TIMELINE	22.09.2026 - 14.10.2026

Scope of Audit

`contracts/aave-v3/AaveV3SupplyBase.sol` `contracts/aave-v3/AaveV3Supply.sol`
`contracts/aave-v3/AaveV3SupplyWithMerkle.sol` `contracts/UniswapV3SwapRouter02Adapter.sol`
`src/CowProtocolAdapter.sol` `src/CowOrderLane.sol` `src/OwnerImmutable.sol`
`src/libraries/GPv2Order.sol` `contracts/CowProtocolModule.sol` `contracts/Container.sol`
`contracts/ContainerLocal.sol` `contracts/ContainerPrincipal.sol`
`contracts/ReshufflingGateway.sol` `contracts/StrategyContainer.sol` `contracts/Vault.sol`
`contracts/StrategyTemplate.sol` `contracts/libraries/StrategyStateLib.sol`

Versions Log

Shift DeFi Strategies

DATE	COMMIT HASH	NOTE
22.09.2026	f87f146cf6ec9e000bd0cdfb8c8cef0f420f4be7	Initial Commit
13.10.2026	bb13c5f676ee6d750ae1fc613efd17b3c13a130f	Re-audit Commit

Shift DeFi Swap Adapters

DATE	COMMIT HASH	NOTE
22.09.2026	3e47d624d1c8084544d16c40622e5dc6666c9e76	Diff Base
22.09.2026	23867125edc0250dc978574237d71bb0af2ccba5	Initial Commit
13.10.2026	9ffe22b1b8bb401fee719f3a85818aa3f1534f15	Re-audit Commit
14.10.2026	ecd6fdfedc767fff550be7569fc0283558f99ab0	Re-audit Remediation Commimt

Shift DeFi CoW Protocol Adapter

DATE	COMMIT HASH	NOTE
22.09.2026	ff1d0bd04c8b3b28fff0aaf23aba15c4d63b1df8	Initial Commit

Shift DeFi Platform

DATE	COMMIT HASH	NOTE
22.09.2026	30f27218e890a5c69b53b93d64c5fce52fbe8294	Diff Base
22.09.2026	9e37173be3cb74d9a9f078acbf23063f304fc80b	Initial Commit
13.10.2026	de125393beb41d479b19c5b4f9a3938ac463a1b9	Re-audit Commit

Mainnet Deployments

At this stage, the contracts have not yet been deployed. Verification that the deployed contracts correspond to the audited code will be conducted once the client provides the addresses of the deployed contracts.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2 Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● FIXED - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● PARTIALLY FIXED - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● ACKNOWLEDGED - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count

0	0	1	7	8
CRITICAL	HIGH	MEDIUM	LOW	TOTAL

Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
M-1	MorphoVault reinvestment stores a pre-fee baseline, so every reinvesting harvest undercharges the fee	MEDIUM	FIXED
L-1	A 1-wei USDC donation makes harvest's Aave supply revert, stalling Vault batches	LOW	FIXED
L-2	aToken fee transfer reverts during an Aave pause, blocking exits from the token state	LOW	FIXED
L-3	At a 100% fee, harvest keeps a pre-fee checkpoint, exempting later income	LOW	FIXED
L-4	Partial fill leaves input and router allowance on the adapter for anyone to take	LOW	FIXED
L-5	Order retirement needs sell-token calls, so an issuer freeze locks batches until an upgrade	LOW	ACKNOWLEDGED
L-6	ReshufflingGateway.blacklistToken requires a zero balance, so any 1-wei transfer blocks delisting	LOW	FIXED
L-7	Native MON on SwapRouter02 makes the Uniswap adapter revert, stalling harvest and Vault batches	LOW	FIXED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

M-1 MorphoVault reinvestment stores a pre-fee baseline, so every reinvesting harvest undercharges the fee

MEDIUM

FIXED

DESCRIPTION

MorphoVault.sol:232-274 `_harvest` and MorphoVault.sol:277-339 `manualClaim` reinvest rewards through `_enterMorphoVault`, which sets `lastAssetsValue` to the value of the whole share balance before the fee shares are transferred to the treasury (MorphoVault.sol:176-178). The final update then compares the post-fee value with that same stored value and writes only if it is greater (MorphoVault.sol:271-272, MorphoVault.sol:336-337). After any non-zero fee the post-fee value is lower, so `lastAssetsValue` keeps the pre-fee figure. `MorphoVault` is outside the scope of this audit; the issue was found while reviewing the same pattern in `AaveV3SupplyWithMerkle`.

The next yield must first cover the value of the transferred fee shares before any of it is charged, and this repeats on every harvest or claim that reinvests. For example, at a 10% fee, a harvest of 10 of yield and 100 of reinvested rewards transfers fee shares worth 11 but leaves the baseline 11 above the position, so the next 11 of yield is charged nothing. The treasury therefore loses about `feePct` of its fee on every reinvesting cycle, 10% of the fee revenue at a 10% fee, for as long as the strategy reinvests rewards.

The impact is Low: users lose nothing and no funds are locked, while the treasury receives only part of the configured performance fee. The likelihood is High: the loss occurs on every reinvesting harvest or claim at any non-zero fee, without any trigger. Low impact with High likelihood gives a severity of Medium.

RECOMMENDATION

> We recommend caching `lastAssetsValue` at the start of `_harvest` and `manualClaim` and ending both by writing the post-fee value when it exceeds the cached value, and the cached value otherwise.

2.4 Low

L-1 A 1-wei USDC donation makes harvest's Aave supply revert, stalling Vault batches

LOW

FIXED

DESCRIPTION

In the supplied state, AaveV3SupplyWithMerkle.sol:104-111 `_harvest` supplies the whole idle USDC balance to Aave, and the supply helper skips only a zero balance (AaveV3SupplyBase.sol:108-120). Aave V3.7, which the Monad pool runs, reverts a supply that mints zero scaled aTokens (ValidationLogic.sol:45). At the current Monad USDC index, a 1-wei supply mints zero. Anyone can therefore make harvest revert by sending 1 wei of USDC to the strategy. The container harvests before every enter and exit (StrategyContainer.sol:273, StrategyContainer.sol:376), so deposit and withdrawal batches of the whole Vault stall until someone sends one more wei. The attacker can repeat this after every harvest.

RECOMMENDATION

> We recommend making `_enterAaveReserveSupplied` skip a supply that would mint zero scaled aTokens or is below a small dust threshold, leaving the USDC for the next harvest.

L-2 aToken fee transfer reverts during an Aave pause, blocking exits from the token state

LOW

FIXED

DESCRIPTION

AaveV3Supply.sol:39-48 and AaveV3SupplyWithMerkle.sol:100-102 charge the performance fee on aToken growth in every strategy state and pay it as an aToken transfer to the treasury (AaveV3Supply.sol:46-48, AaveV3SupplyWithMerkle.sol:120-122). Aave rejects aToken transfers while the reserve is paused (ValidationLogic.sol:436-438). A strategy that the emergency roles have already moved to its token state holds USDC and should keep exiting without Aave, but anyone can send it aTokens before a pause, and their interest keeps accruing while the reserve is paused. During the pause, the harvest that the container runs before every exit (StrategyContainer.sol:376) tries to transfer the fee on that interest and reverts, so every withdrawal batch reverts. The strategy manager can unblock it only by setting the container-wide `feePct` to 0 (StrategyContainer.sol:107-109), which suspends the fee for every strategy of the container. While the reserve is paused, `emergencyExit` should not be attempted either: it cannot withdraw, but it puts the container into emergency resolution (StrategyContainer.sol:395-406), which blocks batch registration and reports for the whole container (ContainerLocal.sol:49-51) until the NAV is accepted.

RECOMMENDATION

> We recommend making the aToken fee transfer best-effort in both `_harvest` implementations: skip it while the reserve is paused and leave `lastReserveATokenBalance` unchanged, so that a later harvest charges the fee.

L-3 At a 100% fee, harvest keeps a pre-fee checkpoint, exempting later income

LOW

FIXED

DESCRIPTION

AaveV3SupplyWithMerkle.sol:93-128 `_harvest` reinvests swapped rewards through `_enterAaveReserveSupplied`, which sets `lastReserveATokenBalance` to the aToken balance before the fee is paid (AaveV3SupplyBase.sol:119). The final write replaces that value only if the post-fee balance exceeds the checkpoint cached at the start of the harvest (AaveV3SupplyWithMerkle.sol:126-127). `StrategyContainer` accepts a `feePct` equal to `MAX_BPS` (StrategyContainer.sol:131-132). At that rate the treasury receives the whole growth and the whole reinvested amount, the post-fee balance equals the cached checkpoint, and the pre-fee value stays in storage. For example, with 1,000 aTokens, 10 of interest and 100 of reinvested rewards, the checkpoint stays at 1,110 while the balance returns to 1,000, so the next 110 of interest is charged no fee, even after `feePct` is lowered.

RECOMMENDATION

> We recommend ending `_harvest` by overwriting the value that `_enterAaveReserveSupplied` wrote: with the post-fee balance when it exceeds the cached checkpoint, and with the cached checkpoint otherwise.

L-4 Partial fill leaves input and router allowance on the adapter for anyone to take

LOW

FIXED

DESCRIPTION

UniswapV3SwapRouter02Adapter.sol:148-157 `swap` pulls `amountIn`, approves `SwapRouter02` for it and calls `exactInput`, but it neither resets the allowance nor returns unspent input. `SwapRouter02` consumes less than `amountIn` when a pool on the path runs out of liquidity. The leftover then stays on the adapter together with a matching allowance. `swap` is permissionless (UniswapV3SwapRouter02Adapter.sol:138-145), so anyone can call it with a worthless `tokenIn` and a whitelisted path that starts with the leftover token. The router then pulls the leftover under that allowance and sends the output to the caller's `receiver`. Running into insufficient liquidity on a whitelisted route is unlikely.

RECOMMENDATION

> We recommend requiring `tokenIn` to equal the first token of `data`, resetting the `SwapRouter02` allowance after `exactInput`, and returning any unspent `tokenIn` to the caller.

L-5 Order retirement needs sell-token calls, so an issuer freeze locks batches until an upgrade

LOW

ACKNOWLEDGED

DESCRIPTION

CowProtocolAdapter.sol:131-149 `cancelOrder` and the filled-order release (CowProtocolAdapter.sol:277-287) remove an order from the pending set only in the same call as the lane's `approveRelayer(sellToken, 0)` and `drain(sellToken)` (CowOrderLane.sol:48-66). Nothing else removes it. If the sell token's issuer freezes the lane or pauses the token, `drain` reverts, and so does `approveRelayer` for tokens whose `approve` also checks the freeze, such as USDC, because `forceApprove` retries a failed approval and bubbles its revert. The order can then be neither cancelled nor resolved; examples are a USDC or USDT blacklist and a PYUSD or RLUSD freeze. Reports, `setCowAdapter` and the Vault's reshuffling toggles require an empty pending set (CowProtocolModule.sol:135-143), so the Vault batch in progress stays blocked. A stuck gateway order also keeps the Vault in reshuffling mode, where `withdraw()` reverts (Vault.sol:524). Only an upgrade of the consumer's proxy or the issuer lifting the freeze ends the lock.

RECOMMENDATION

> We recommend skipping `approveRelayer(token, 0)` when the lane's allowance is already zero, and adding an owner-only path that invalidates the order at settlement, marks it cancelled, frees its lane and removes it from the pending set, attempting the allowance reset and the drain in `try/catch`.

CLIENT'S COMMENTARY

Acknowledged

L-6 ReshufflingGateway.blacklistToken requires a zero balance, so any 1-wei transfer blocks delisting

LOW

FIXED

DESCRIPTION

ReshufflingGateway.sol:112-122 `blacklistToken` delists a token only while the gateway holds none of it, as documented for the gateway (IReshufflingGateway.sol:61-62). However, every function that moves tokens out of the gateway requires Vault reshuffling mode (ReshufflingGateway.sol:168-170). Any address can therefore block the delisting of a whitelisted token by sending 1 wei of it to the gateway. A transfer to a gateway lane whose filled order `blacklistToken` drains before the check has the same effect. The Safe can still delist the token in one privately submitted transaction while the Vault is idle: it enables reshuffling mode, moves the exact balance out, calls `blacklistToken` and disables the mode. No funds are at risk, but the gateway can keep using the token until it is delisted.

RECOMMENDATION

> We recommend adding a per-token dust threshold that `TOKEN_MANAGER_ROLE` sets and `blacklistToken` compares against, and transferring any residual balance at delisting to a fixed recovery address that no balance check reads.

L-7 Native MON on SwapRouter02 makes the Uniswap adapter revert, stalling harvest and Vault batches

LOW

FIXED

DESCRIPTION

UniswapV3SwapRouter02Adapter.sol:156-167 `swap` reverts with `PartialFill` unless its `tokenIn` balance falls by exactly `amountIn`. For a `WETH9` input, `SwapRouter02` pays the pool from its own native balance when that balance is large enough and does not take the adapter's tokens (PeripheryPayments.sol:58-61). On Monad `WETH9` is WMON, the reward token that `AaveV3SupplyWithMerkle._harvest` sells through this adapter (AaveV3SupplyWithMerkle.sol:109-112). Anyone can send 1 wei of WMON to the strategy and leave 1 wei of MON on `SwapRouter02` through any payable router call, so the next harvest's swap is paid by the router and reverts. `mustSucceed = false` does not prevent this: `SwapRouter.tryPredefinedSwap` returns `(false, 0)` only when no route is set and otherwise calls the adapter without catching its revert (SwapRouter.sol:87-111, StrategyTemplate.sol:645-646). The container harvests before every enter and exit (StrategyContainer.sol:273, StrategyContainer.sol:376), so every batch that includes the strategy reverts. Anyone can remove the parked MON with `refundETH()`, but the attacker can repeat this after every harvest for 2 wei.

RECOMMENDATION

> We recommend that `swap`, when `tokenIn` is the router's `WETH9`, first call `refundETH()` on the router, wrap any native amount received and add it to the swap input, accepting native transfers only from `SwapRouter02`, so that the router always pulls the adapter's tokens.

[3 ABOUT MIXBYTES]

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io